

ICMP Smurf Attack — Simulation & Defense

CSE 406 · Computer Security Sessional · Docker-based isolated lab

ICMP reflection/amplification (DDoS)

Docker Compose

Scapy · iptables · tcpdump

4 defenses tested

One-line summary: This project builds a small isolated virtual network, launches a classic *Smurf* denial-of-service attack inside it (where one attacker packet is multiplied into many packets aimed at a victim), and then demonstrates four different defenses that each shut the attack down — all measured with real packet captures.

1. What is a Smurf Attack? (the core idea)

A Smurf attack is a **reflection + amplification** denial-of-service attack that abuses ICMP (ping).

It relies on two tricks used together:

1. **Source-address spoofing** — the attacker forges the "From" address on a ping request, writing the *victim's* IP address as the sender instead of their own.
2. **Directed broadcast** — the attacker sends that ping not to one host, but to a network's *broadcast address* (e.g. `203.0.113.255`), which delivers it to *every* host on that network at once.

The result: every host on the broadcast network ("amplifiers") receives a ping request that appears to come from the victim, so they all send their ping *replies* to the victim. One packet from the attacker becomes **N packets** hitting the victim, where N is the number of amplifier hosts. Scale N up and the victim is flooded — that is the denial of service.

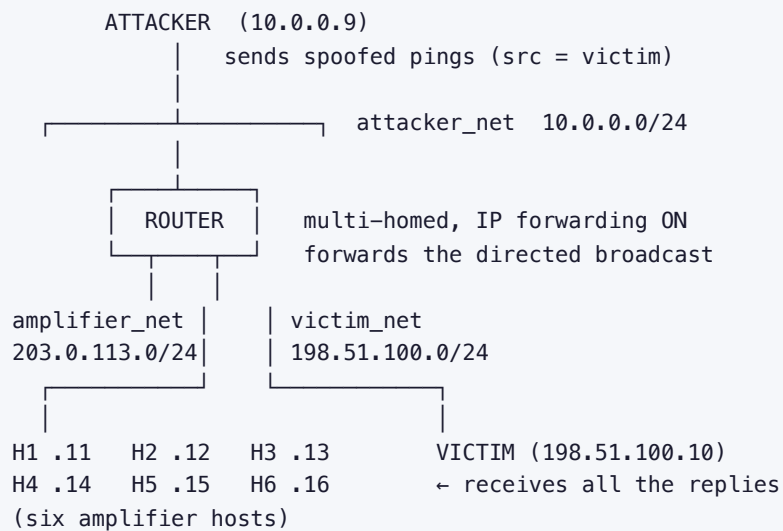
Why it's called "amplification": the attacker spends little bandwidth (a few small packets) but the victim absorbs a much larger volume. In our lab the amplification factor is **6×** (6 amplifier hosts). On the real 1990s Internet it could be hundreds or thousands.

2. Project Goal

- Recreate the full Smurf attack chain safely in an **isolated lab** (no real networks touched).
- **Measure** the amplification with packet captures at the victim.
- Implement and **verify four defenses**, showing each one reduces the attack traffic to zero.
- Produce **analysis and plots** comparing attack vs. defenses.

3. The Lab Topology

Everything runs as Docker containers on three isolated virtual networks, connected by one multi-homed **router**. Addresses use RFC 5737 "documentation" ranges (safe, non-routable in practice).



Role	Container(s)	Address	Job
Attacker	attacker	10.0.0.9	Generates spoofed ICMP packets (Scapy)
Router	router	.2 on each net	Routes between all 3 subnets; where most defenses live
Amplifiers	amp-h1 ... amp-h6	203.0.113.11-16	Reply to broadcast pings → reflect to victim
Victim	victim	198.51.100.10	Target; captures & analyzes the flood

4. How the Attack Works — step by step

1. The attacker runs `smurf_attack.py`, which crafts ICMP Echo Requests with a **forged source IP = the victim (198.51.100.10)** and a **destination = the amplifier broadcast (203.0.113.255)**.
2. The packet travels to the router. The router **forwards the directed broadcast** onto the amplifier network, delivering it to all six amplifier hosts.
3. Each amplifier sees a ping "from the victim" and dutifully sends an **Echo Reply back to the victim**.
4. The victim is hit by **6 replies for every 1 packet** the attacker sent. Repeating this at speed floods the victim.

Measured result (from packet captures at the victim):

10

SPOOFED PACKETS
SENT

6

AMPLIFIER HOSTS
REPLY

60

REPLIES HIT THE
VICTIM

6x

AMPLIFICATION
FACTOR

The victim's capture shows **60 Echo Replies from 6 unique sources** — proof the amplification worked.

5. The Four Defenses

Each defense is enabled one at a time, the same attack is re-run, and the victim's capture is checked. In every case the reflected traffic reaching the victim drops to **zero**.

#	Defense	Runs on	How it works	Standard
1	Disable directed-broadcast forwarding	Router	Router drops any packet aimed at the broadcast address (<code>203.0.113.255</code>), so the ping never fans out to the amplifiers. <i>The root-cause fix.</i>	RFC 2644
2	Ingress filtering (source validation)	Router	Router checks that a packet's source address matches the network it arrived on (reverse-path filtering + iptables). The spoofed "from victim" packet arriving on the attacker network is rejected.	BCP 38 / RFC 2827
3	Suppress broadcast echo replies	Amplifiers	Each host sets <code>icmp_echo_ignore_broadcasts = 1</code> , so it simply ignores pings sent to a broadcast address — removing it from the reflection pool.	§10.2
4	ICMP rate limiting	Router	iptables caps how many ICMP Echo Replies per second may pass. A mitigation (limits impact) rather than a cure — the reflection path still exists.	§10.2

Defense strength, ranked: Defenses 1 & 2 are the strongest because they stop the attack at its two enabling conditions (broadcast + spoofing). Defense 3 removes the amplifiers. Defense 4 only softens the blow. A real network would combine them (defense in depth).

6. Results Summary

Scenario	Replies reaching victim	Outcome
Baseline (normal ICMP)	~0 at victim	Normal operation
Smurf attack (no defense)	60 (6× amplification)	Victim flooded
Defense 1 — broadcast disabled	0	Attack blocked
Defense 2 — ingress filtering	0	Attack blocked
Defense 3 — suppress echo	0	Attack blocked
Defense 4 — rate limiting	0 / capped	Impact reduced

Plots are generated automatically in `results/` : per-capture timelines, replies-per-amplifier charts, and a `defense_comparison.png` bar chart contrasting all scenarios.

7. How to Run It (demo commands)

```
# 1. Build and start the whole lab (9 containers)
docker compose up -d --build

# 2. Run the complete experiment: baseline → attack → all 4 defenses → analysis
./run_experiment.sh all

# Or run individual phases:
./run_experiment.sh baseline # normal ICMP
./run_experiment.sh attack   # the Smurf attack
./run_experiment.sh defense  # test all four defenses
./run_experiment.sh analyze   # analysis + plots

# 3. Tear everything down
docker compose down
```

Key files

File	Purpose
docker-compose.yml	Defines the 9 containers and 3 networks
scripts/attacker/smurf_attack.py	Crafts the spoofed ICMP packets (Scapy)
scripts/victim/capture.sh	tcpdump capture on the victim
scripts/defense/*.sh	The four defense scripts
scripts/analysis/*.py	Packet analysis & plotting
run_experiment.sh	Orchestrates the whole demo

8. Key Technical Points (things the examiner may probe)

- **Why does the victim get flooded and not the attacker?** Because the source IP is spoofed to the victim, so the amplifiers reply to the victim, not the true sender.
- **Why is a broadcast used?** To reach many hosts with one packet — that is where the amplification comes from.
- **Why is this "reflection"?** The victim never talks to the attacker; the amplifiers are innocent third parties who reflect traffic, hiding the attacker.
- **Directed-broadcast forwarding** had to be explicitly enabled on the router (`net.ipv4.conf.*.bc_forwarding=1`) because modern Linux drops directed broadcasts by default — which is itself Defense 1 in action.
- **Isolation:** the lab has no route to the internet (the router carries no default route), so the attack cannot escape — important for doing this safely and ethically.

9. Likely Q&A

Q: Is the Smurf attack still a threat today?

A: Rarely in its original form — modern routers disable directed-broadcast forwarding by default (RFC 2644) and many networks deploy ingress filtering (BCP 38). But the underlying idea — reflection & amplification — is very much alive in DNS, NTP, and memcached amplification DDoS attacks.

Q: What is the single most effective defense?

A: Disabling directed-broadcast forwarding on routers (Defense 1) kills this specific attack at the source. Ingress filtering (Defense 2) is the most important general defense because it stops source spoofing, which underlies almost all reflection attacks.

Q: How did you prove the attack and defenses actually work?

A: With packet captures (tcpdump) taken on the victim. Under attack we captured 60 reflected replies from 6 amplifiers; with each defense enabled the victim captured 0 — objective, measured evidence rather than assumption.

Q: Why Docker instead of real machines or VMs?

A: Lightweight, reproducible, and fully isolated. Nine "hosts" and three networks run on one laptop, the whole topology is code (docker-compose.yml), and it can be torn down and rebuilt identically in seconds.

Q: Why do the amplifiers respond to a broadcast ping at all?

A: By default a host answers ICMP Echo Requests, including those sent to its network's broadcast address. Defense 3 turns that behavior off.